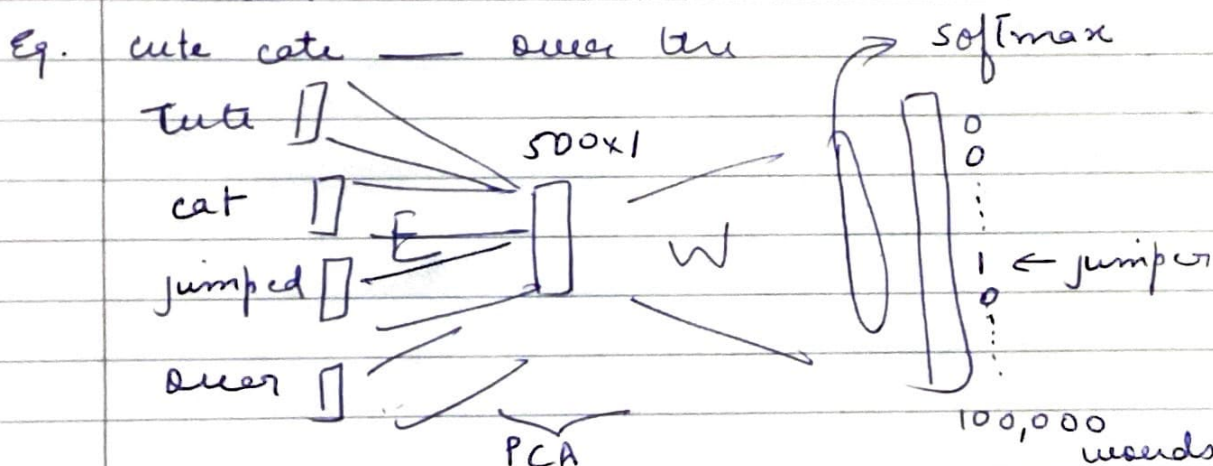


Week 10 - Speech Tech

L1 Intro to Word2Vec

- more from a text processing perspective
- it was able to find the context btw. 2 words. It was able to find the relationships.
- 2 disadv. of 1-hot.
 - (i) not very efficient. Creates sparse matrices
 - (ii) each word is equidistant ($\sqrt{2}$) to each other.



initialize E, W & then train it on multiple
egs. learn the params E & W .

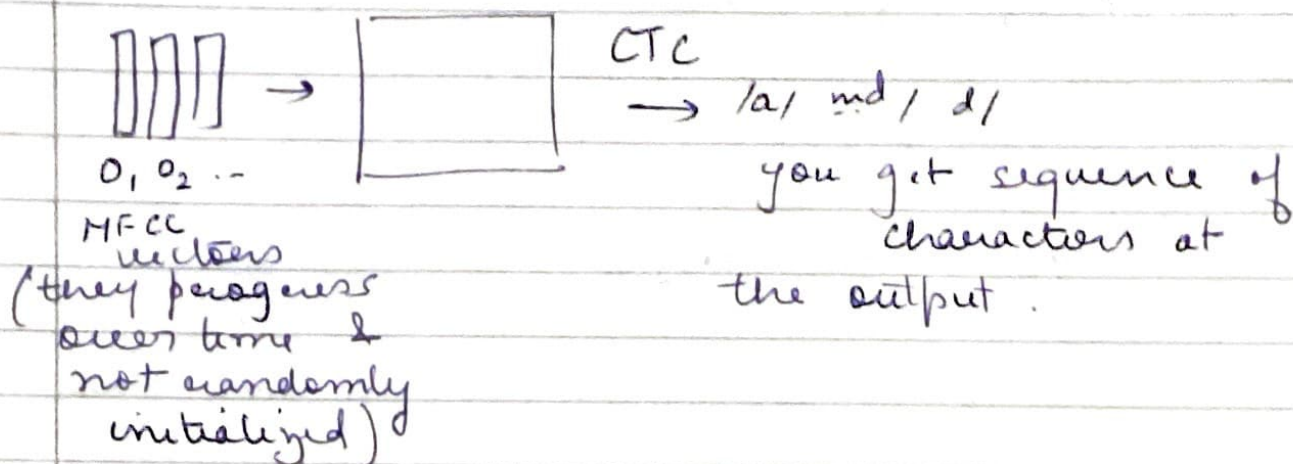
$$e_{the}^{500 \times 1} = E^{500 \times 100000} \cdot \frac{1}{the}$$

- also, it was able to make some connect
btw. 2 words.
- word2vec $\rightarrow$ use one-hot vectors & during
learning the embedding matrix, it doesn't
require any humanly labelled data.

- $\therefore$, word2vec is a self-supervised learner.

L2 Sequence-to-sequence modeling

- ASR problem

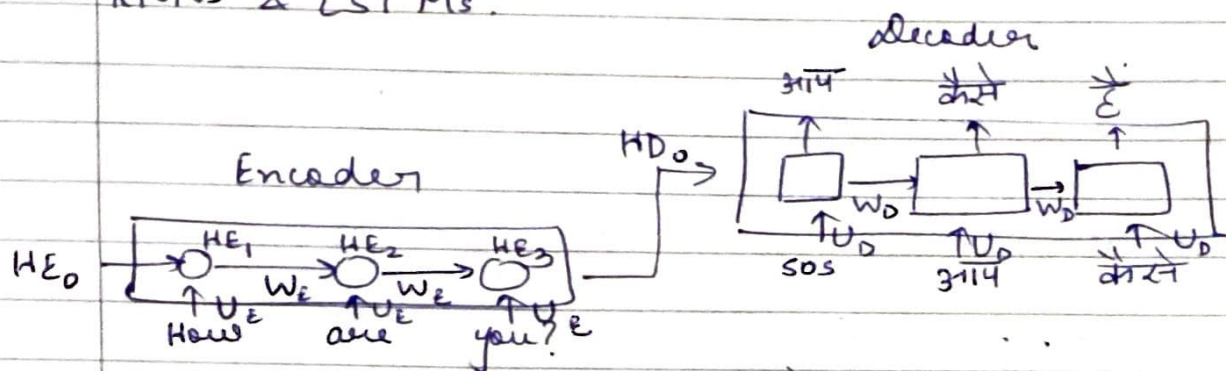


- In CTC,
the $\text{input len} \geq \text{output len}$

Why, because CTC might lead to deduplication of characters extracted from ASR sys. Otherwise, each feature will correspond to 1 character only.

- in text processing, some sequence-to-sequence examples, could be question-answering or translation tasks. It would be sequence-of-words at both input & output level.
- For the translation tasks, the output lang. has to be built the separate embedding matrix for that output language.

- the early seq. models were based on RNNs & LSTMs.



$$HE_t = f(U_E X_t + W_E HE_{t-1})$$

$$HD_t = f(U_D Y_{t-1} + W_D HD_{t-1})$$

$$Y_t = f(U_D H_{Dt})$$

- the network got trained on huge volumes of trainable data.
- we can do for ASR as well.

seq. of acoustic vectors $\rightarrow$ seq. of words.

$\hookrightarrow$ we will use LSTMs here, because to avoid the vanishing / exploding gradient.

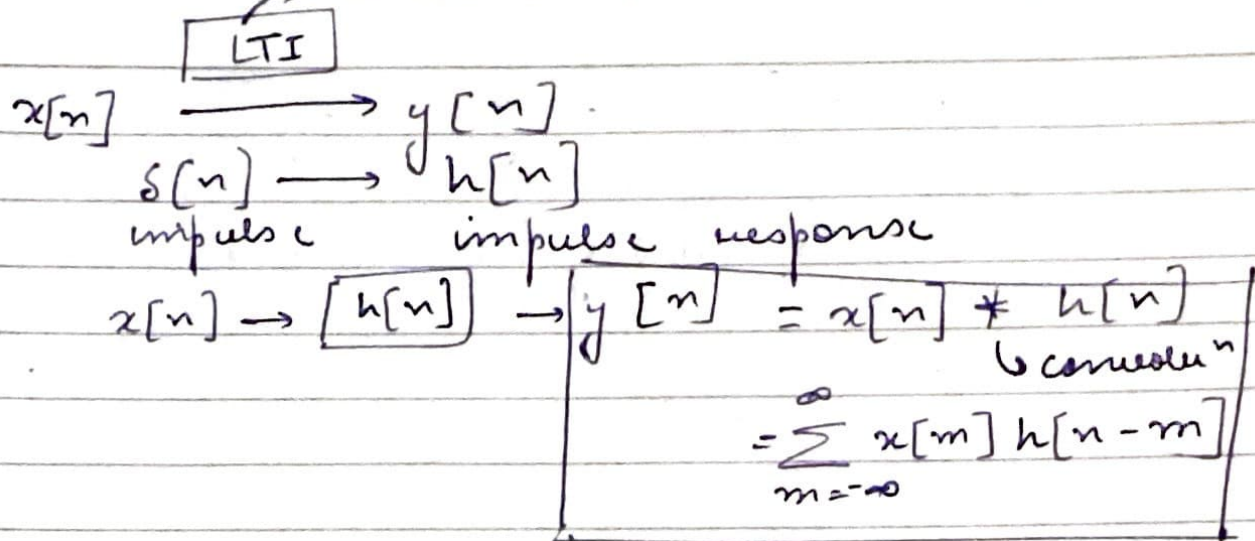
- Surely, we can improve on this - we can add the context in decoder for every input so that it does not dies out.
- or, process all the encoder hidden states dynamically $\Rightarrow$ attention mechanisms.

L3

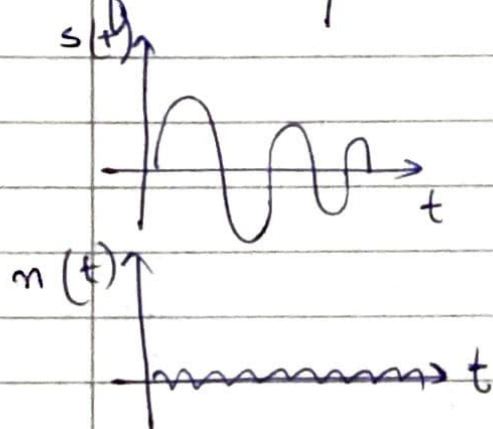
Intro to CNN

- works well on images. now used in speech as well.

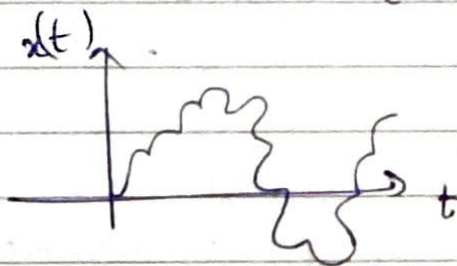
linear time invariant



- filtering $\rightarrow$ removing noise but keeps few things



$$x(t) = s(t) + n(t)$$



$$x[n] \rightarrow \text{LTI cleaning sys.} \rightarrow y[n] = \frac{1}{2M+1} \sum_{m=-M}^M x[n-m]$$

$\hookrightarrow M+1$

smoothing process by taking an avg.

$n(t) \rightarrow$ almost becomes 0

$s(t) \rightarrow$ becomes smoother by removing noise.

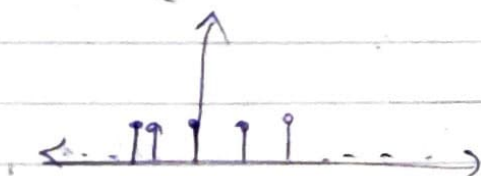
L4 CNN Example

filtering to remove noise (low pass)
small freq.

$$x[n] \rightarrow \boxed{} \rightarrow y[n] = \frac{1}{2M+1} \sum_{m=-M}^M x[n-m]$$

removes rapidly happening frequency.

$$h[n] = \begin{cases} \frac{1}{2M+1} & -M \leq n \leq M \\ 0 & \text{otherwise} \end{cases} \quad \left. \begin{array}{l} \text{coefficients} \\ \text{of the} \\ \text{filter} \end{array} \right\}$$



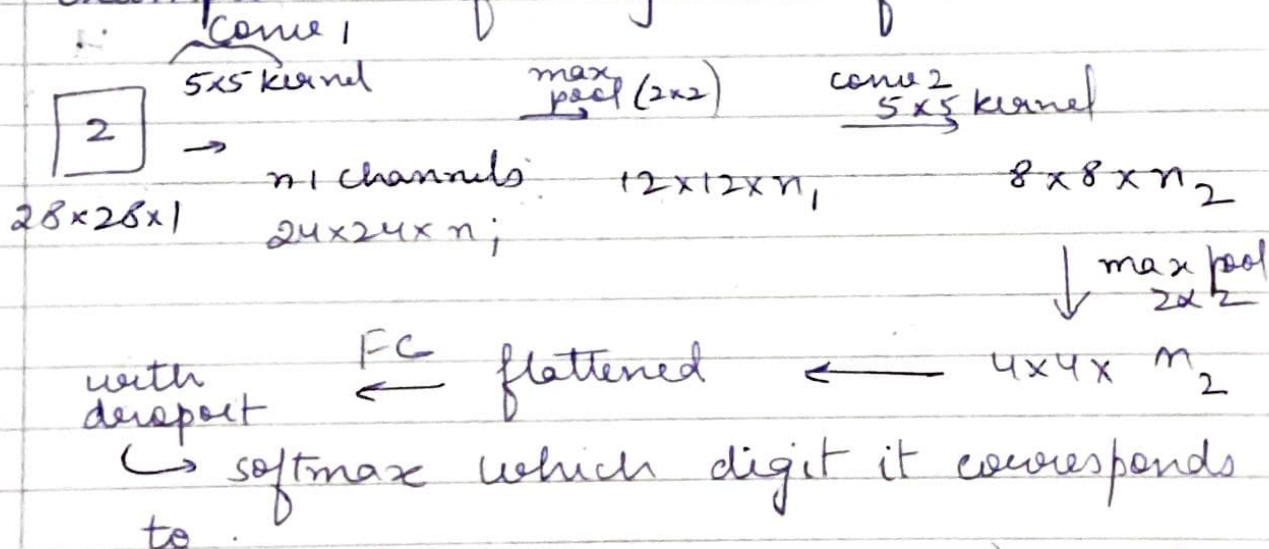
$$y[n] = h[n] * x[n] \\ = \sum h[m] x[n-m]$$

$h[n] \rightarrow$ filter, $*$ $\rightarrow$ convolution

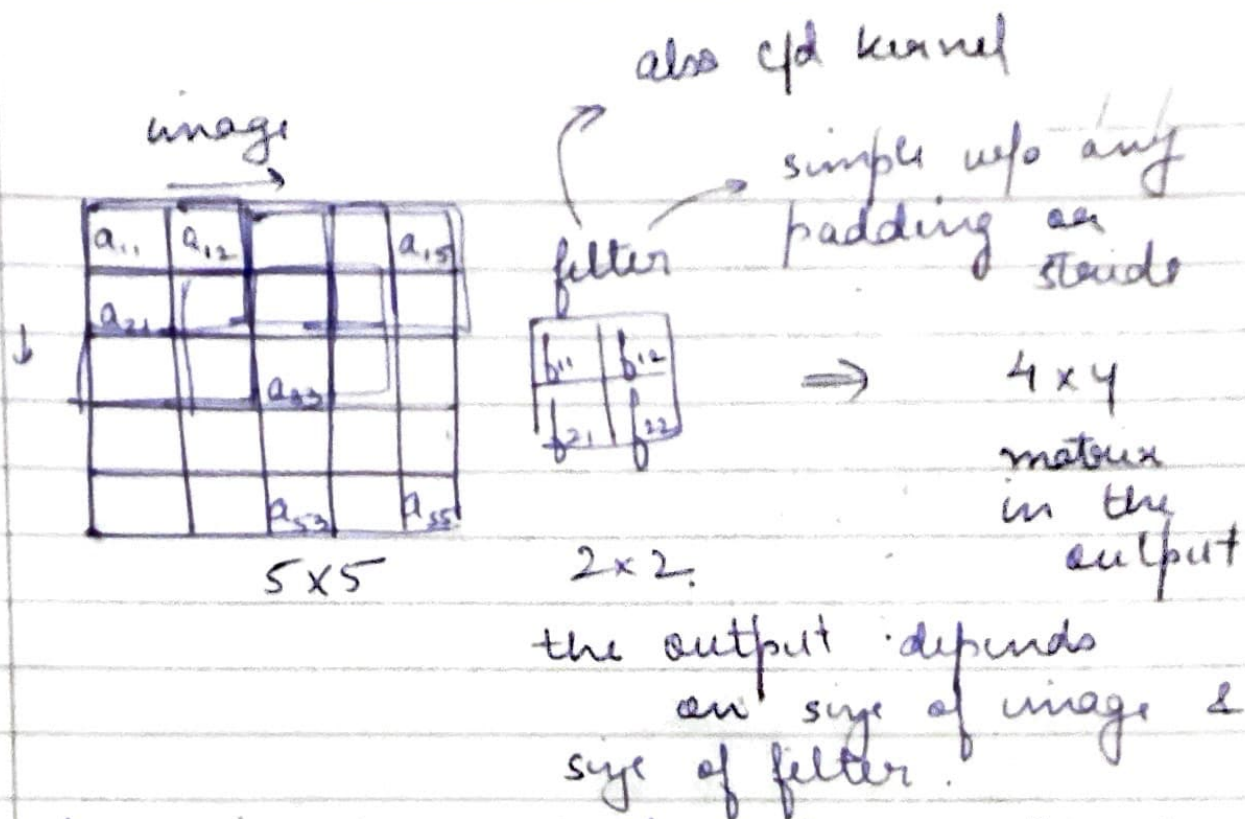
$x[n] \rightarrow$ currently depends on 1D.

but for images it is a 2D operation.

- example CNN for digit classification



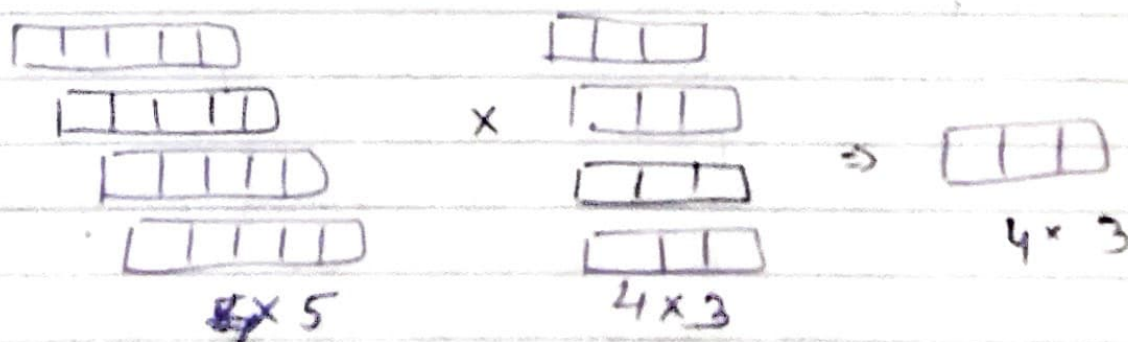
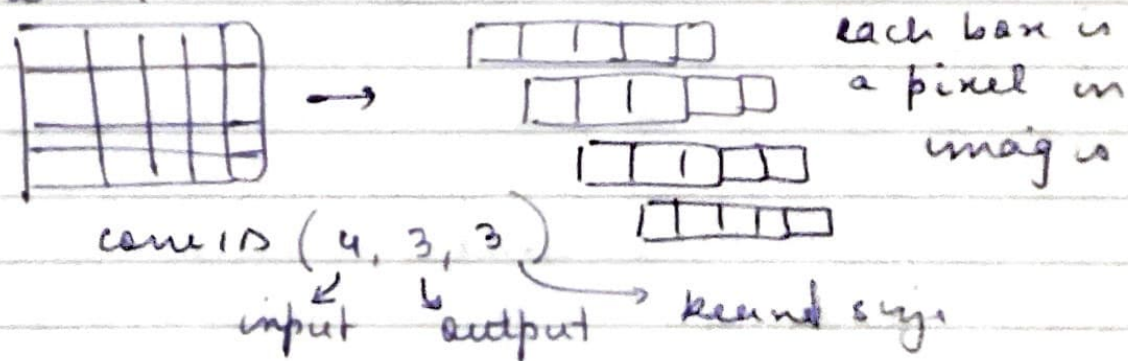
- we take multiple filters (n_1 & n_2) why?
to learn different kinds of features about the image.



- flattening helps to look like a FFNN.

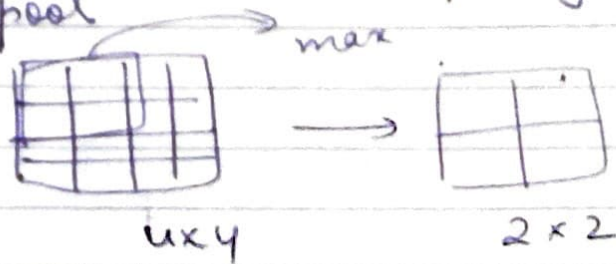
L5 CNN for Speech

- convoluⁿ, kernel, stride (step), padding (0.), pooling (avg, max), dilation, flattening (like FFNN), fully connected networks
- conv 1D.



In conv 1D, filters move in 1 direction only

- we use padding, when we want to maintain the dimensions.
- stride means the step-size.
- max pool



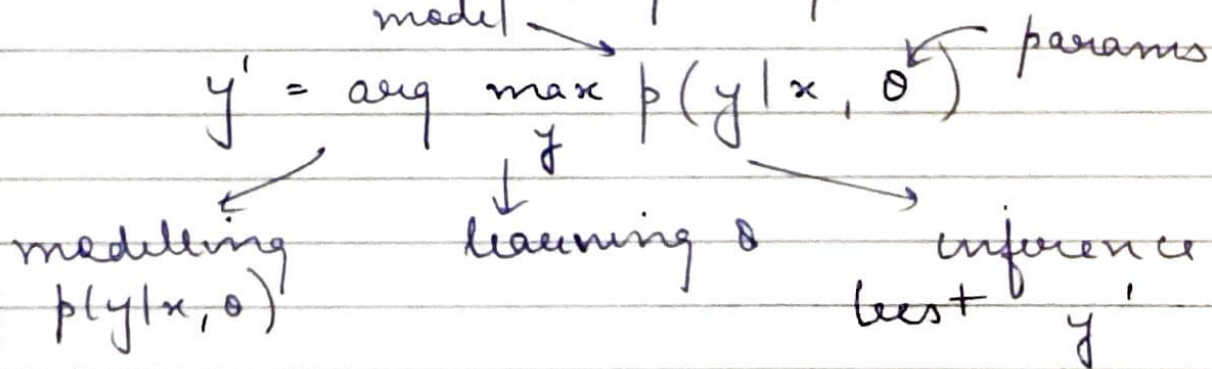
- dilate $\rightarrow$ specifies the way in which the kernel is applied.
- similarly, in videos (timestamps) we can apply conv 3D.
- in Transformer $\rightarrow$ parallelizable training
 $\hookrightarrow$ each convoluⁿ is looking at its own neighbours, therefore, they all can be parallelized.

L6

Sequence to Sequence Networks

- RNN $\rightarrow$ Recurrence (models sequential data)
 $\hookrightarrow$ addⁿ of recurrence through hidden layers provides seq memory needed. The info. in the hidden state is a representaⁿ of past inputs.
- RNN problem $\rightarrow$ vanishing / exploding gradient
 $\hookrightarrow$ solve using LSTM.
 $\hookrightarrow$ also, it is time-consuming as each input is taken 1-by-1. Solution - Transformer

- after long time stamps, RNNs tends to forget what was initially fed into the system. As their influence decreases, as the time steps increases. So, the intent of the Statement has to be understood.
- This short-term memory & vanishing gradient is due to the nature of back-propagation. Bigger the gradient, bigger the adjustments in weights. If gradient goes to 0, no update will happen. If gradient is too large, then huge jumps will occur resulting non-leading to global minima.
- All this gets solved using LSTMs. They have gates to fine-tune the flow of info.
- Machine translation (seq 2 seq)



- for encoder & decoder, we have to use 2 RNNs $\rightarrow$ for both of them.

